

Loan Approval Prediction

Preprocessing, class imbalance, model comparison & deployment threshold | Dataset: Kaggle - bhanupratapbiswas/loan-approval-prediction-case-study

» **Notebook:** [View on GitHub: Task2/Loan Approval Prediction.ipynb](#)

Executive Summary

After imputing missing values, encoding categoricals and scaling numeric features, four models were compared using SMOTE and class-weighting to address the 68.7% / 31.3% class imbalance. Logistic Regression with `class_weight='balanced'` achieved the best ROC-AUC (0.85), ahead of SMOTE-based Logistic Regression, Decision Tree and Random Forest. Credit_History alone is a strong, explainable signal -- applicants with a credit history are approved 79.6% of the time vs. just 7.9% without one -- and sits among the top 3 model features alongside ApplicantIncome and LoanAmount, which are closely bunched at the top of the Random Forest's feature-importance ranking.

614

Applicants analyzed

0.85

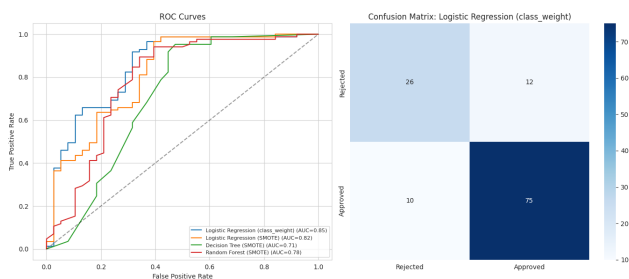
Best model ROC-AUC

79.6%

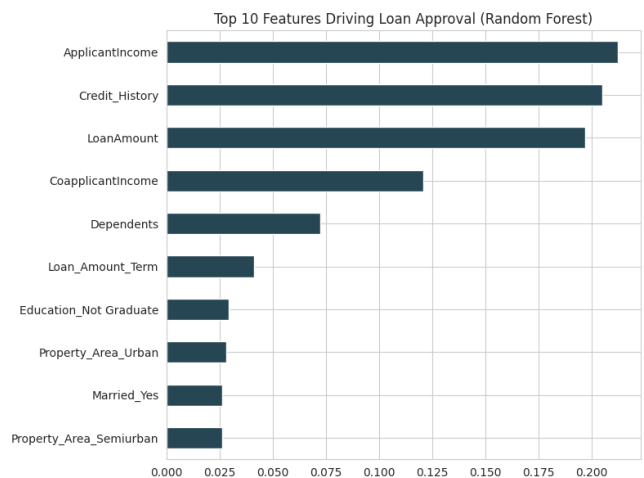
Approval rate w/ credit history

7.9%

Approval rate w/o credit history



ROC curves and confusion matrix (best model)



Top drivers of loan approval (Random Forest)

Key Findings

- Class imbalance: 68.7% approved vs 31.3% rejected -- addressed via SMOTE and `class_weight='balanced'`.
- Best model: Logistic Regression (`class_weight`) -- ROC-AUC 0.85, ahead of SMOTE variants and tree models.
- Credit_History alone predicts well (79.6% approval with vs 7.9% without), but in the full Random Forest it's closely bunched with ApplicantIncome and LoanAmount, not a lone dominant lever.
- F1-maximizing decision threshold on the test set differs from the default 0.5 -- see notebook Section 8.
- Given the small sample (614 rows), validate the chosen threshold on more data before production use.

Top Recommendations

- **Lead underwriting with credit history:** It's a strong, explainable signal on its own -- prioritize collecting it early, alongside income and loan amount which rank comparably in the model.
- **Set threshold above 0.5 for risk-conscious deployment:** Bad-approval downside typically exceeds good-rejection downside; bias the cutoff accordingly.
- **Prefer class_weight over SMOTE at this sample size:** SMOTE's synthetic points are more prone to overfitting on only 614 rows.
- **Re-validate on more data before go-live:** Precision/recall estimates carry real uncertainty at this sample size.
- **Monitor real-world default rates post-launch:** Use them to retune the threshold rather than freezing it at the F1-optimal point.